

Scheduling Asynchronous Inference in Robotic Systems: From Action Chunking to Real-Time VLA Execution

WANG, Ruixiang

The Chinese University of Hong Kong, Shenzhen
April 2026

Abstract

Robotic systems increasingly rely on large neural policies that map rich observations and language instructions to continuous control. This trend has made inference latency a first-class systems problem. A high-frequency controller may require an action every 20 to 70 ms, while modern vision-language-action (VLA), diffusion, or flow-matching policies may require hundreds of milliseconds to generate a new action chunk. If the robot waits for the model, it pauses. If it executes stale commands, it becomes less reactive. If a new chunk is switched in naively, the boundary between old and new commands can produce discontinuous, jerky, and sometimes unsafe motion. The central thesis of this report is that action chunking should be treated not only as a policy representation technique, but also as a runtime scheduling interface between slow inference and real-time robot execution.

This thesis-style report studies three papers as a sequential research line. First, *Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware* introduces Action Chunking with Transformers (ACT), a policy interface that predicts a sequence of future actions instead of a single next action [18]. ACT reduces effective decision horizon, improves temporal consistency, and enables low-cost bimanual manipulation on contact-rich tasks. Second, *Real-Time Execution of Action Chunking Flow Policies* formulates the latency problem as an inference-time scheduling and inpainting problem [2]. Its real-time chunking (RTC) algorithm overlaps execution of the current chunk with generation of the next chunk, freezes committed actions, and inpaints the mutable future suffix. Third, *Real-Time Robot Execution with Masked Action Chunking* argues that asynchronous execution introduces not only inter-chunk discontinuity but also intra-chunk inconsistency between current perception and partially executed actions [17]. REMAC therefore trains the policy to tolerate masked, delayed, and partially committed chunks, while preserving prefixes at sampling time.

The three papers together define a thesis topic: scheduling asynchronous inference in robotic systems. ACT defines the unit of scheduling, namely the action chunk. RTC defines the runtime scheduling mechanism, namely a producer-consumer loop over a committed prefix and mutable suffix. REMAC defines the policy adaptation needed when runtime scheduling changes the distribution seen by the controller. The report develops this correlation explicitly, reviews related work in imitation learning, VLA policies, diffusion and flow policies, and robot runtime systems, and synthesizes performance results across simulation and real-world manipulation. The main conclusion is that future robot learning systems need cross-layer co-design: policy representations must expose schedulable temporal structure, runtimes must reason about deadlines and commitment, and training procedures must anticipate the observation-action mismatch created by asynchronous inference.

Contents

Abstract	iii
1 Introduction	1
1.1 Motivation	1
1.2 Research Questions	2
1.3 Thesis Stack	2
1.4 Contributions of This Report	3
1.5 Thesis Outline	3
2 Background and Related Work	5
2.1 Imitation Learning and Compounding Error	5
2.2 Action Chunking as a Temporal Interface	5
2.3 Diffusion, Flow, and VLA Policies	6
2.4 Asynchronous Inference in Robot Deployment	6
2.5 Runtime Systems Versus Learned-Policy Scheduling	7
2.6 VLA Inference as a Service	7
2.7 Why the Three-Paper Line Is Narrow but Sufficient	8
2.8 Summary	9
3 Paper 1: ACT	11
3.1 Paper Overview	11
3.2 Problem Setting	11
3.3 Action Chunking	12
3.4 Model Architecture	13
3.5 Temporal Ensembling as Proto-Scheduling	13
3.6 A Worked ACT Example	14
3.7 Performance Evidence	14
3.8 Limitations for Real-Time VLA Execution	14
3.9 Contribution to the Thesis	16
4 Paper 2: RTC	17
4.1 Paper Overview	17
4.2 The Latency Model	17
4.3 Failure of Naive Asynchronous Execution	18
4.4 Freeze and Inpaint	18
4.5 Soft Masking	20
4.6 Runtime Interpretation	20
4.7 A Concrete Timeline Example	21
4.8 Algorithmic View	22

4.9	Evaluation	22
4.10	Latency Cost	22
4.11	Contribution to the Thesis	25
5	Paper 3: REMAC	27
5.1	Paper Overview	27
5.2	Inter-Chunk and Intra-Chunk Failure Modes	27
5.3	Masked Action Chunking	28
5.4	Components of REMAC	28
5.5	Prefix-Preserved Sampling	30
5.6	Evaluation in Simulation	30
5.7	Evaluation on Real Robots	31
5.8	Contribution to the Thesis	32
6	Performance Evaluation and Synthesis	33
6.1	Evaluation Methodology	33
6.2	Representation Versus Scheduling Versus Robustness	33
6.3	A Unified Runtime Model	34
6.4	Performance Lessons	34
6.5	What the Evaluations Do Not Yet Resolve	35
6.6	A Proposed Evaluation Trace	35
6.7	Scheduling Design Rules	36
6.8	How the Three Papers Form a Thesis	36
7	Open Problems	39
7.1	Deadline-Aware Action Queue Scheduling	39
7.2	Adaptive Prediction and Execution Horizons	39
7.3	Multi-Policy and Multi-Rate Scheduling	39
7.4	Observability and Runtime Debugging	39
7.5	Safety and Formal Guarantees	40
8	Conclusion and Future Work	41
8.1	Summary of Contributions	41
8.2	Future Work	41
8.3	Final Statement	41

List of Figures

2.1	SmolVLA’s asynchronous inference stack, reproduced as a cropped paper figure. The diagram is useful related work because it makes the queue and remote policy server explicit, even though SmolVLA is not the central paper of this thesis.	7
2.2	SmolVLA’s synchronous-versus-asynchronous evaluation, reproduced as a cropped paper figure. The result is included as related evidence that asynchronous inference can reduce completion time while maintaining similar success rates.	8
3.1	The ACT/ALOHA system and example manipulation skills, reproduced as a cropped figure from the ACT paper. The figure is the empirical starting point for the thesis: action chunking is introduced to make fine-grained real-world manipulation learnable on low-cost hardware.	11
3.2	ACT architecture: a CVAE-style transformer policy maps observations and joint states to a future action sequence. The cropped figure shows that the basic output object is a chunk, not a single command.	12
3.3	Action chunking and temporal ensembling in ACT. Multiple chunks can predict the same timestep, making overlap aggregation a natural operation before asynchronous scheduling is introduced.	13
3.4	ACT ablation and baseline results, reproduced as a cropped paper figure. The results support the claim that action chunking and temporal ensembling improve fine-grained manipulation compared with one-step baselines. . . .	15
4.1	RTC motivation figure, reproduced as a cropped paper figure. It contrasts real-time chunking with synchronous execution and emphasizes that smooth trajectories must be maintained even when inference delay is substantial. . .	17
4.2	RTC’s bifurcation example, reproduced as a cropped paper figure. A newly generated chunk can choose a different motion mode from the currently executing chunk, creating discontinuity when switched in late.	19
4.3	RTC guidance diagram, reproduced as a cropped paper figure. The committed prefix receives full guidance, the intermediate overlap receives soft guidance, and the future suffix remains flexible.	19
4.4	Hard and soft masking in RTC, reproduced as a cropped paper figure. Soft masking preserves cross-chunk continuity more effectively than hard in-painting of the committed prefix alone.	20
4.5	RTC pseudocode area, reproduced as a cropped paper figure. The algorithmic view reinforces that RTC is a runtime loop coordinating controller ticks, inference calls, and queue updates.	22

4.6	RTC Kinetix evaluation, reproduced as a cropped paper figure. The plots show robustness to inference delay and improved use of shorter execution horizons compared with baselines.	23
4.7	RTC real-world evaluation, reproduced as a cropped paper figure. The results compare cumulative task progress and throughput under synchronous, temporal ensembling, and real-time chunking execution.	23
4.8	RTC latency tables, reproduced as a cropped appendix page. The tables show both the benefit and cost of RTC: it avoids execution stalls but adds inference-time overhead.	24
5.1	REMAC's execution-paradigm figure, reproduced as a cropped paper figure. The figure distinguishes synchronous execution, asynchronous execution, inter-chunk discontinuity, and intra-chunk inconsistency.	27
5.2	REMAC appendix figures showing robot setup, scheduling-function variants, and task examples. The cropped page adds deployment context for the real-world evaluation.	29
5.3	REMAC simulation results, reproduced as a cropped paper figure. The figure reports solve rates and execution time across Kinetix tasks under different inference delays.	30
5.4	REMAC component ablations and integration with test-time methods, reproduced as a cropped paper figure. The table and schedule comparison show how masked action chunking improves delay robustness.	31
5.5	REMAC real-robot completion progress and delay-injection result, reproduced as a cropped paper figure. The results show higher progress under delay, especially on the harder task.	31
5.6	REMAC visual and kinematic comparison under injected delay, reproduced as a cropped paper figure. The figure shows faster completion and smoother velocity/acceleration profiles.	32

List of Tables

1.1	Mapping the three papers to the thesis stack.	2
2.1	Related-work taxonomy for asynchronous inference in robot systems. . . .	9
3.1	Temporal ensembling in ACT interpreted as a runtime policy.	14
3.2	ACT through the lens of asynchronous inference scheduling.	15
4.1	Runtime variables in RTC.	18
4.2	RTC as an operating-systems-style scheduling algorithm.	21
4.3	Worked example of prefix commitment under delayed inference.	21
5.1	Comparison between RTC and REMAC.	28
5.2	REMAC components and their role in the thesis stack.	30
6.1	Evaluation focus across the three main papers.	33
6.2	Unified scheduling state for action-chunking robot policies.	34
6.3	Qualitative performance synthesis.	35
6.4	Proposed trace fields for evaluating asynchronous robot inference schedulers.	36

Chapter 1

Introduction

1.1 Motivation

Robotics has entered a regime in which control policies are no longer small functions called inside a fixed real-time loop. Modern robot policies increasingly contain image encoders, language encoders, diffusion or flow-matching decoders, transformer action heads, and sometimes remote GPU inference services. These policies are attractive because they can represent high-dimensional perception, language-conditioned intent, and multi-modal action distributions. They are also expensive. In a physical control loop, expensive inference is not merely a throughput problem. It changes what the robot does while the policy is thinking.

The mismatch can be described with a simple timing picture. A low-level controller samples at a fixed period Δt . At every tick it needs a command a_t . A large policy, however, may require a delay δ to produce a chunk of future actions. If $\delta < \Delta t$, synchronous closed-loop control is viable: observe, infer, execute, and repeat. If δ is comparable to multiple controller periods, the robot must either stop while waiting, continue executing an older plan, or switch to a new plan after several old actions have already been committed. All three choices have consequences. Waiting creates idle gaps and reduces task throughput. Continuing an old plan makes the robot less reactive to new observations. Switching to a new plan can create discontinuities because the newly generated action chunk may have been sampled from a different mode than the old one.

The topic of this report is therefore not simply “robot learning” and not simply “real-time operating systems.” It sits between them. The operating-system view is useful because the central objects are buffers, deadlines, producer-consumer queues, and committed versus mutable work. The robot-learning view is indispensable because the queued object is not a generic task; it is an action trajectory produced by a learned policy whose errors are shaped by the training distribution. Scheduling asynchronous inference in robotic systems is the problem of deciding when to invoke an expensive policy, which actions are already committed to execution, how to splice a newly generated future into an old trajectory, and how to train the policy so that such splicing remains inside distribution.

Action chunking is the key abstraction. Instead of predicting one action at a time, an action-chunking policy predicts

$$A_t = [a_t, a_{t+1}, \dots, a_{t+H-1}],$$

where H is the prediction horizon. At first glance this is a learning trick: predicting a local trajectory makes imitation learning smoother and reduces the effective horizon. But in modern VLA systems it becomes a runtime interface. A chunk can be queued, partially consumed, overlapped with a newly generated chunk, or repaired by inpainting. This report

follows how the idea develops from representation to scheduling to robustness.

1.2 Research Questions

The thesis is organized around three research questions. They correspond to three layers of the asynchronous inference stack.

1. **RQ1:** What policy representation exposes enough temporal structure for precise robot manipulation while remaining learnable from demonstrations?
2. **RQ2:** Given an action-chunking policy whose inference is slower than the controller, how should a runtime schedule inference and execution without creating pauses or discontinuities?
3. **RQ3:** How should the policy be trained or adapted when asynchronous execution creates a mismatch between current perception and partially committed actions?

The three main papers answer these questions sequentially. ACT answers **RQ1** by turning the action into a chunk. RTC answers **RQ2** by turning chunk generation into an asynchronous inpainting problem. REMAC answers **RQ3** by making the policy robust to masked, delayed, and prefix-conditioned chunks. This is the correlation that makes the three papers suitable for a thesis-style report rather than a disconnected literature survey.

1.3 Thesis Stack

The thesis stack used in this report has three layers. The first layer is the policy representation layer. It defines what is predicted and how the prediction is conditioned on observation. The second layer is the runtime scheduling layer. It defines the action queue, the inference job, the execution horizon, and the deadline at which a generated chunk becomes available. The third layer is the robustness layer. It defines how the model is trained or adapted so that the asynchronous execution protocol does not create out-of-distribution inputs at deployment.

Table 1.1: Mapping the three papers to the thesis stack.

Layer	Representative paper	Main object	Thesis contribution
Policy representation	ACT	Future action sequence A_t	Defines the schedulable unit.
Runtime scheduling	RTC	Action queue, frozen prefix, inpainted suffix	Makes chunked execution real time.
Robustness and adaptation	REMAC	Masked action chunk and prefix-preserved sampling	Makes asynchronous chunks reliable under mismatch.

This stack deliberately avoids treating ROS2 or real-time operating systems as the primary literature. ROS2 executors, DDS middleware, and RTOS kernels are important in deployed robots, but the papers in this report focus on a more specific systems issue: the scheduling of learned action-chunk inference. The relevant deadline is not only a kernel scheduling deadline. It is also a semantic deadline: after some actions have been consumed, the prefix of the new chunk can no longer be changed without contradicting what the robot has already done.

1.4 Contributions of This Report

This report makes four contributions in the style of a pre-oral Ph.D. thesis.

C1 - A scheduling interpretation of action chunking. The report reframes action chunking from a technique for imitation learning into a scheduling interface. This interpretation explains why the same object, an action sequence, is useful for both reducing compounding error and hiding inference latency.

C2 - A sequential analysis of ACT, RTC, and REMAC. The three papers are discussed separately and in order. Each chapter explains the problem setting, method, evidence, limitations, and thesis relevance of one paper.

C3 - A cross-paper evaluation synthesis. The report compares what is measured in each paper: ACT emphasizes task success and precision manipulation, RTC emphasizes throughput and delay robustness, and REMAC emphasizes completion progress under perception-action mismatch.

C4 - A research agenda for OS-style robot inference scheduling. The report extracts open systems problems, including deadline-aware action queues, adaptive execution horizons, multi-policy arbitration, runtime observability, and training-time exposure to scheduler decisions.

1.5 Thesis Outline

Chapter 2 reviews background and related work in imitation learning, action chunking, diffusion and flow policies, VLA policies, and asynchronous robot deployment. Chapter 3 studies ACT as the origin point for chunked action prediction in fine-grained manipulation. Chapter 4 studies RTC as a runtime algorithm for real-time asynchronous execution of chunking flow policies. Chapter 5 studies REMAC as a training-time and sampling-time response to the mismatch created by asynchronous inference. Chapter 6 provides a performance evaluation and cross-paper synthesis. Chapter 7 concludes with the main takeaways and future work.

Background and Related Work

2.1 Imitation Learning and Compounding Error

Imitation learning maps demonstrations to a policy. In robotic manipulation, a demonstration is usually a trajectory of observations and actions. Behavioral cloning trains a policy to predict the demonstrator action at each state. This simple formulation has two weaknesses. First, the learned policy is evaluated on states induced by its own previous actions, not only on states from the demonstration distribution. Small errors can therefore compound over time. Second, human demonstrations are not perfectly Markovian. They may contain pauses, hesitations, correlated sub-motions, or small corrective actions whose meaning is distributed across time rather than present in a single state-action pair. Prior work on multi-modal imitation learning, such as Behavior Transformers, also shows that demonstration data can contain several valid modes that are hard to represent with a one-step deterministic policy [15].

The fine manipulation tasks emphasized by ACT make these weaknesses visible. Threading a cable tie, opening a cup, inserting a battery, or manipulating a zipper all require contact-rich, temporally coherent motion. A one-step policy may produce locally plausible commands that are globally incoherent. For example, the gripper may approach the object correctly but fail to preserve a mode of interaction long enough to complete the task. Action chunking addresses this by predicting a short future trajectory. It reduces the number of high-level decisions and gives the policy a local temporal object to model.

2.2 Action Chunking as a Temporal Interface

An action chunking policy predicts a horizon- H action sequence. During rollout the controller may execute the entire sequence, execute the first s actions and replan, or combine overlapping predictions through temporal ensembling. This introduces two horizons. The prediction horizon H describes how far into the future the model imagines. The execution horizon s describes how many actions are consumed before the next policy update is used. A small s makes the system more reactive, while a large s reduces the number of inference calls. BID later studies the same reactivity-consistency tension from a test-time decoding perspective, making it a useful bridge between ACT-style chunking and RTC-style runtime correction [11].

This distinction is central for asynchronous inference. In a synchronous system, the policy is called only after the controller is ready for a new chunk. In an asynchronous system, inference can begin before the current chunk is exhausted. The runtime therefore has a buffer of pending actions and a background inference job. Some actions in the overlapping

region are already guaranteed to execute. Others can still be replaced by a new chunk. This creates the committed-prefix and mutable-suffix structure used by RTC and REMAC.

2.3 Diffusion, Flow, and VLA Policies

Diffusion policies and flow-matching policies are attractive for robot control because they can represent multi-modal action distributions. Instead of outputting a single deterministic command, the model iteratively denoises or transports a noisy action sequence toward a plausible trajectory conditioned on observation. This is useful when multiple strategies are valid, such as going around an obstacle from above or below. It is also computationally expensive because sampling requires multiple network evaluations or integration steps. Diffusion Policy is the most direct predecessor in this family, showing that action diffusion can be effective for visuomotor manipulation but also motivating the latency concerns addressed by RTC and REMAC [5].

Vision-language-action policies add another cost. They condition the action model on images, language instructions, and robot state. The promise is generality: a single model can respond to different instructions and embodiments. RT-1, RT-2, PaLM-E, and OpenVLA show how language and large-scale visual pretraining can be connected to robot action generation [3, 4, 6, 10]. The systems price is latency. Image preprocessing, VLM encoding, action expert computation, communication between robot and server, and chunk aggregation all contribute to the time between observation and executable command. As a result, the policy no longer behaves like a lightweight real-time callback. It behaves like a service whose outputs must be buffered and scheduled.

2.4 Asynchronous Inference in Robot Deployment

Asynchronous inference decouples perception and action prediction from action execution. The robot executes actions from a queue while the policy computes future actions in parallel. This is analogous to prefetching in computer systems: the runtime attempts to ensure that the consumer, the controller, never starves. However, action queues differ from ordinary data queues. The order, smoothness, and semantic compatibility of actions matter. Replacing the queue with a new chunk is not like replacing bytes in a buffer. A discontinuity may physically perturb the robot, break contact, or push the policy into an unseen state.

SmolVLA provides a useful deployment example even though it is not one of the three main papers [16]. Its asynchronous inference stack separates a RobotClient from a PolicyServer and uses an action queue threshold to decide when to request more actions. The design demonstrates that asynchronous inference can improve responsiveness and reduce completion time, but it also highlights why a principled scheduling algorithm is necessary. Queue management, observation filtering, and overlap aggregation are runtime decisions that directly affect policy behavior. Large robot datasets such as Open X-Embodiment and DROID further reinforce this deployment view: if policies are trained from broad heterogeneous data, then runtime systems must preserve enough temporal structure to make such policies usable on real robots [13, 9].

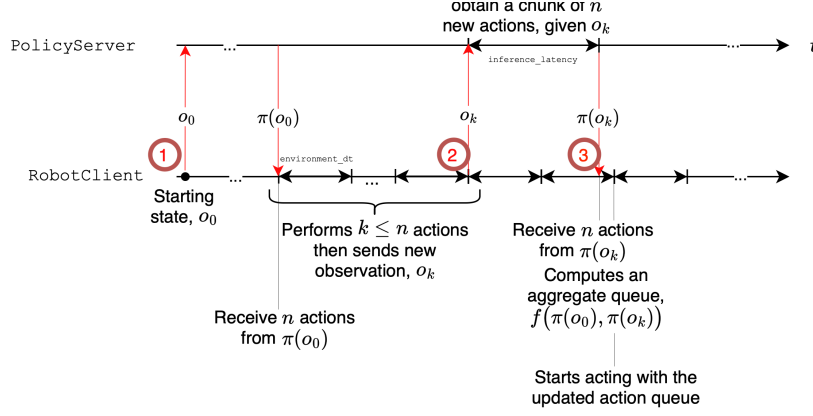


Figure 2 | Asynchronous inference. Illustration of the asynchronous inference stack. Note that the policy can be run on a remote server, possibly with GPUs.

A less resource-intensive approach is to entirely exhaust the chunk $\mathbf{A}$ before predicting a new chunk of actions, a strategy we refer to as *synchronous* (sync) inference. Moreover, sync inference efficiently allocates computation every n timestep, resulting in a reduced average computational burden at control time. In contrast, it inherently hinders the responsiveness of robot systems, introducing blind lags due to the robot being *idle* while computing $\mathbf{A}$.

We directly assess the lack of adaptiveness of robot systems due to acting open-loop, and the presence of lags at runtime by decoupling action chunk prediction $\mathbf{A}$ from action execution $a_t \leftarrow \text{POPFRONT}(\mathbf{A}_t)$, developing an *asynchronous* (async) inference stack (Algorithm 1), whereby a **ROBOTCLIENT** sends an observation o_t to a **POLICYSERVER**, receiving an action chunk $\mathbf{A}_t$ once inference is complete (Figure 2). In this, we avoid execution lags by triggering chunk prediction while the control loop is still consuming a previously available queue, aggregating it with the newly incoming

Figure 2.1: SmolVLA’s asynchronous inference stack, reproduced as a cropped paper figure. The diagram is useful related work because it makes the queue and remote policy server explicit, even though SmolVLA is not the central paper of this thesis.

2.5 Runtime Systems Versus Learned-Policy Scheduling

Traditional robot systems literature often studies middleware, real-time kernels, executor scheduling, control-loop jitter, and priority inversion. These problems remain important, but the focus here is narrower and more learning-aware. The scheduler must understand that an action chunk has a temporal support. The prefix may be committed, the suffix may be mutable, and the policy may require conditioning on old chunks to avoid discontinuity.

This is why the thesis does not choose ROS2 or RTOS papers as the main line. Those papers would make the report primarily about CPU tasks and message passing. The ACT-RTC-REMAC line makes the report about the scheduling interface between learned inference and physical execution. The operating-system analogy is still present, but the scheduling object is policy-generated motion.

2.6 VLA Inference as a Service

Another useful way to understand the related work is to treat the policy as a service. In many emerging robot systems, the robot computer performs sensing and low-level execution, while a GPU workstation or remote server performs policy inference. The service boundary introduces communication latency, batching opportunities, device contention, and failure modes familiar from distributed systems. However, the service output is not a text token or a classification label. It is a sequence of continuous actions whose timing is physically meaningful.

This service view clarifies the difference between throughput and control quality. A

policy server may maximize average throughput by batching or by running larger models, but the robot controller cares about deadline misses and queue starvation. A policy server may return a high-quality chunk, but if it arrives after the controller has already consumed the relevant prefix, the runtime must decide which part of the result is still usable. The best inference service for robotics is therefore not only the fastest model. It is the model-runtime pair that produces usable future actions before their deadlines.

Policy	VLA pt.	Success Rate (%) – Real World			
		Pick-Place	Stacking	Sorting	Avg.
Single-task Training					
SmolVLA (0.45B)	No	55	45	20	40
Multi-task Training					
SmolVLA (0.45B)	No	80	40	35	51.7
SmolVLA (0.45B)	Yes	75	90	70	78.3

Table 5 | Effect of pretraining and multitask learning. Success rate (%) across three tasks using SmolVLA trained in multi-task and single-task settings. Results with SO100 robot.

Inference	Success Rate (%) – Real World				Inference	Time (s) – Real World			Inference	# of Cubes – Real World		
	Pick-Place	Stacking	Sorting	Avg		Total	Avg	Std		Total	Avg	Std
Sync	75	90	70	78.3	Sync	137.5	13.75	2.42	Sync	9	1.8	0.45
Async	80	90	50	73.3	Async	97.0	9.70	2.95	Async	19	3.8	1.3

(a) | Performance (success rates). (b) | Task completion time. (c) | Performance in fixed time.

Figure 5 | Comparison between synchronous (Sync) and asynchronous (Async) inference. We evaluate SmolVLA on three real-world tasks under both inference modes. Asynchronous inference achieves similar success rates (left) but is significantly faster (middle) and complete more tasks (right) in fixed-time settings. Hyperparameters have been optimized for Pick-Place and reused across tasks.

Figure 2.2: SmolVLA’s synchronous-versus-asynchronous evaluation, reproduced as a cropped paper figure. The result is included as related evidence that asynchronous inference can reduce completion time while maintaining similar success rates.

2.7 Why the Three-Paper Line Is Narrow but Sufficient

A broad survey of robot learning could include many additional papers and systems: Diffusion Policy, RT-1, RT-2, PaLM-E, Octo, OpenVLA, π_0 , ALOHA, Mobile ALOHA, and several real-time robot middleware systems [5, 3, 4, 6, 12, 10, 1, 7]. This report uses them only as background because the requested thesis topic is more specific. The topic is not “general robot foundation models.” It is the scheduling of asynchronous inference once a model already produces action chunks.

The selected three papers cover the minimal complete chain. ACT introduces the representation that makes chunk-level scheduling possible. RTC shows how to schedule that representation at inference time when the model is slow. REMAC shows how training must change after the scheduler changes the distribution of perception and action prefixes. Removing any one paper breaks the chain. Without ACT, there is no chunk interface. Without RTC, there is no explicit runtime scheduling mechanism. Without REMAC, there is no answer to the distribution shift induced by the scheduling mechanism.

Table 2.1: Related-work taxonomy for asynchronous inference in robot systems.

Research area	Main concern	Relevance to this thesis
Behavioral cloning	Learn actions from demonstrations	Motivates compounding-error reduction.
Action chunking	Predict local action trajectories	Provides schedulable action buffers.
Diffusion/flow policies	Sample multi-modal action chunks	Increases inference latency and mode-switch risk.
VLA deployment	Combine language, vision, and action	Makes remote and asynchronous inference realistic.
Robot middleware	Schedule CPU callbacks and messages	Supplies concepts but not policy semantics.
Runtime policy re-pair	Maintain continuity under delay	Directly addressed by RTC and REMAC.

2.8 Summary

The related work shows a progression from learning stable action representations to deploying large policies under latency. Action chunking was introduced as a way to make imitation learning more reliable. Diffusion, flow, and VLA policies make the same representation expensive enough that runtime scheduling becomes unavoidable. Asynchronous inference then creates new failure modes that are partly systems-level and partly learning-level. This motivates the three-paper sequence studied in the next chapters.

Chapter 3

Paper 1: ACT

3.1 Paper Overview

The first paper, *Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware*, introduces Action Chunking with Transformers (ACT) [18]. The paper is motivated by a striking observation: fine-grained manipulation does not necessarily require expensive industrial robots if the learning system and teleoperation interface are designed carefully. The authors build ALOHA, a low-cost bimanual teleoperation and robot platform, and demonstrate tasks such as opening a condiment cup, inserting a battery, threading a cable tie, and manipulating tape.

The contribution most relevant to this thesis is not the transformer architecture by itself. It is the control interface. ACT predicts a chunk of future actions rather than a single next action. This turns a high-frequency control problem into a sequence-generation problem. The policy maps the current observation, joint state, and a latent style variable to a future action sequence. During execution, overlapping predictions can be combined through temporal ensembling.

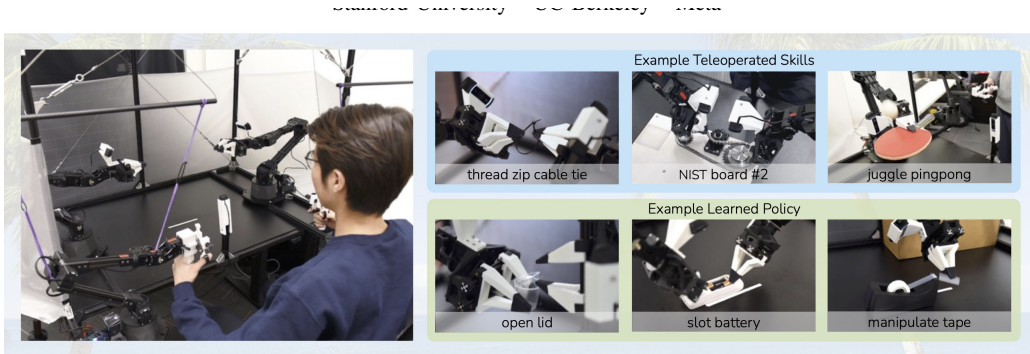


Fig. 1: ALOHA 🌈: A Low-cost Open-source Hardware System for Bimanual Teleoperation. The whole system costs <\$20k with off-the-shelf

Figure 3.1: The ACT/ALOHA system and example manipulation skills, reproduced as a cropped figure from the ACT paper. The figure is the empirical starting point for the thesis: action chunking is introduced to make fine-grained real-world manipulation learnable on low-cost hardware.

3.2 Problem Setting

ACT targets tasks where high precision, contact, and visual feedback interact. Inserting a battery requires aligning a small object with a spring-loaded slot. Opening a cup requires coordinating two arms, stabilizing the cup, and prying the lid without excessive force. Threading a cable tie requires maintaining grasp and orientation across several delicate sub-steps.

These tasks are difficult for scripted controllers because contact conditions vary and visual feedback is essential. They are also difficult for standard behavioral cloning because local errors accumulate.

The paper’s learning setting is end-to-end imitation learning. Demonstrations are collected through a teleoperation interface. Observations include camera images and robot joint states. Actions correspond to future joint targets. The goal is to learn a policy that can reproduce the demonstrated manipulation behavior from a modest number of demonstrations. The central challenge is therefore not only perception, but also temporal coherence.

3.3 Action Chunking

ACT models the policy as

$$\pi_{\theta}(A_t | o_t), \quad A_t = [a_t, a_{t+1}, \dots, a_{t+k-1}],$$

where k is the chunk size. This formulation reduces the effective decision horizon by approximately a factor of k when a full chunk is executed. More importantly, it asks the model to represent local motion primitives. The output is not a point command but a small trajectory. This helps with demonstrations that contain temporally correlated behavior, because the model can encode a short segment of intent.

The paper also uses a temporal ensemble. At a given controller time, several previously generated chunks may contain predictions for the same action. ACT combines them with exponentially decaying weights. Older predictions provide stability, while newer predictions provide reactivity. This idea is an early precursor to the later asynchronous methods. Temporal ensembling already treats overlapping chunks as objects that can be merged. RTC and REMAC later make the overlap explicit in a runtime schedule.

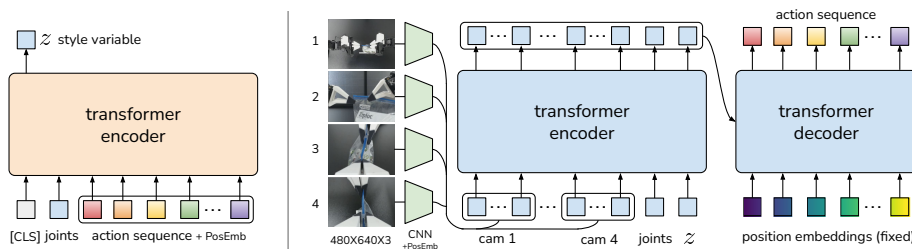


Fig. 4: *Architecture of Action Chunking with Transformers (ACT)*. We train ACT as a Conditional VAE (CVAE), which has an encoder and a decoder. *Left*: The encoder of the CVAE compresses action sequence and joint observation into z , the style variable. The encoder is discarded at test time. *Right*: The decoder or policy of ACT synthesizes images from multiple viewpoints, joint positions, and z with a transformer encoder, and predicts a sequence of actions with a transformer decoder. z is simply set to the mean of the prior (i.e. zero) at test time.



To train ACT on a new task, we first collect human demonstrations using *ALOHA*. We record the joint positions of the leader robots (i.e. input from the human operator) and use them as actions. It is important to use the leader joint positions instead of the follower’s, because the amount of force applied

Figure 3.2: ACT architecture: a CVAE-style transformer policy maps observations and joint states to a future action sequence. The cropped figure shows that the basic output object is a chunk, not a single command.

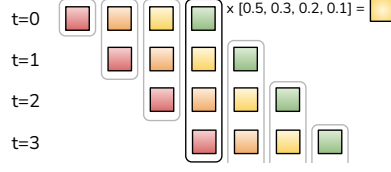


Fig. 5: We employ both Action Chunking and Temporal Ensembling when applying actions, instead of interleaving observing and executing.

able to a single research arm such as Franka Emika Panda. *ALOHA* enables the teleoperation of:

- **Precise tasks** such as threading zip cable ties, picking credit cards out of wallets, and opening or closing ziploc bags.
- **Contact-rich tasks** such as inserting 288-pin RAM into

Figure 3.3: Action chunking and temporal ensembling in ACT. Multiple chunks can predict the same timestep, making overlap aggregation a natural operation before asynchronous scheduling is introduced.

3.4 Model Architecture

ACT uses a conditional variational autoencoder with transformer components. During training, the encoder observes the action sequence and produces a latent style variable. The decoder then predicts the action sequence conditioned on the current observation, joint state, and latent. At test time the encoder is discarded and the decoder generates actions using a fixed or sampled latent. The transformer architecture is useful because it can model dependencies inside the output action sequence.

From a systems perspective, the architecture matters less than the interface it exposes. The policy call produces an H -step object. This object can be stored in a queue, partially consumed, temporally ensembled, or overwritten. In other words, ACT provides the data structure later scheduling algorithms need. Without chunked output, there is no prefix to freeze and no suffix to inpaint.

3.5 Temporal Ensembling as Proto-Scheduling

Temporal ensembling in ACT is usually presented as a smoothing method, but it can also be read as a proto-scheduler. At a timestep t , several previous chunks may contain candidate actions for the same controller tick. The temporal ensemble chooses a weighted combination. This is a scheduling decision over redundant future predictions. Older chunks represent earlier commitments; newer chunks represent fresher observations. The exponential weighting parameter controls the balance between stability and reactivity.

This perspective makes ACT more connected to RTC than it first appears. RTC also reasons over overlapping chunks, but it uses the overlap to constrain generation rather than merely to average outputs. REMAC also reasons over overlapping chunks, but it trains the model to handle the partial validity of those overlaps. Thus, temporal ensembling is the first appearance of an important idea: an action at time t may be predicted multiple times, and the system must decide which prediction should influence execution.

Table 3.1: Temporal ensembling in ACT interpreted as a runtime policy.

Element	Scheduling interpretation
Multiple overlapping chunks	Redundant candidate work for the same future timestep.
Exponential weights	A policy for trading old commitment against new observation.
Small decay parameter	More stable but less reactive execution.
Large decay parameter	More reactive but potentially less smooth execution.
No explicit delay model	The main limitation addressed by RTC.

3.6 A Worked ACT Example

Consider a controller running at 50 Hz, so $\Delta t = 20$ ms. Suppose ACT predicts a chunk of $H = 100$ actions. If the system executes the full chunk before asking for another, the high-level policy runs at 0.5 Hz. This is stable but not reactive. If the system replans every $s = 10$ actions, the policy runs at 5 Hz and produces overlapping chunks. Temporal ensembling can combine these overlaps. In both cases the low-level controller still receives 50 actions per second, but the expensive policy is called much less often than a one-step policy would be called.

The same example reveals the limitation. If the policy call takes 300 ms, then a synchronous replan every $s = 10$ actions is impossible because 10 actions cover only 200 ms. The robot must either wait, use a larger execution horizon, or request the next chunk before the current one is exhausted. This is exactly the transition from ACT to RTC: the chunk size and execution horizon become runtime scheduling parameters rather than only learning hyperparameters.

3.7 Performance Evidence

The ACT paper reports strong performance on several real-world tasks with limited demonstrations. The key qualitative result is that action chunking improves fine manipulation compared with one-step baselines. The paper also reports the benefit of temporal ensembling and shows that high-frequency teleoperation is not always necessary when chunking provides temporal coherence. These results support the first research question: temporal structure in the output representation can make imitation learning more stable.

3.8 Limitations for Real-Time VLA Execution

ACT is not designed primarily as an asynchronous runtime algorithm. If inference is fast relative to the execution horizon, the original loop is sufficient. The robot observes, predicts a chunk, executes actions, and then predicts again. This works when the model is small enough and the chunk boundary does not introduce idle time.

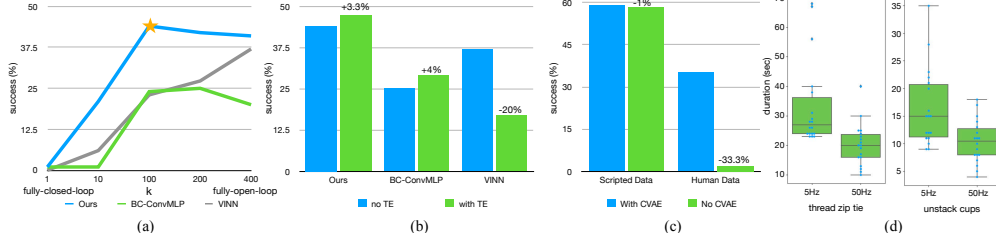


Fig. 8: (a) We augment two baselines with action chunking, with different values of chunk size k on the x-axis, and success rate on the y-axis. Both methods significantly benefit from action chunking, suggesting that it is a generally useful technique. (b) Temporal Ensemble (TE) improves our method and *BC-ConvMLP*, while hurting *VINN*. (c) We compare with and without the CVAE training, showing that it is useful when learning from human data. (d) We plot the distribution of task completion time in our user study, where we task participants to

Figure 3.4: ACT ablation and baseline results, reproduced as a cropped paper figure. The results support the claim that action chunking and temporal ensembling improve fine-grained manipulation compared with one-step baselines.

Table 3.2: ACT through the lens of asynchronous inference scheduling.

Aspect	Interpretation
Policy output	A future action chunk A_t that can be stored, overlapped, or partially executed.
Learning benefit	Reduces effective horizon and captures temporally coherent local behavior.
Runtime assumption	The original execution is still mostly synchronous: observe, infer, execute, and observe again.
Scheduling limitation	If inference becomes slow, a controller can still wait at chunk boundaries or consume stale actions.
Thesis role	Defines the schedulable unit for RTC and REMAC.

The limitation appears when the policy becomes larger. In a VLA or flow-matching policy, generating the next chunk may take hundreds of milliseconds. If the robot waits at the boundary, the trajectory contains pauses. If it starts inference too late, the queue can empty. If it starts inference while executing the current chunk, it must decide how to reconcile old and new chunks. ACT gives the system a chunk but does not specify how to schedule inference or how to handle committed actions.

3.9 Contribution to the Thesis

ACT contributes the representation layer of the thesis. Its core insight is that a policy can output a structured temporal object rather than a single action. That object improves learning and provides a runtime buffer. The rest of the thesis asks what happens when this buffer is produced by a slow model and consumed by a real-time controller.

The most important lesson from ACT is that the boundary between learning and systems is already blurred. The temporal ensemble is a learning-motivated smoothing procedure, but it also resembles runtime aggregation over overlapping chunks. This prepares the conceptual ground for RTC, where overlap is no longer just a smoothing trick. It becomes a scheduling constraint.

Chapter 4

Paper 2: RTC

4.1 Paper Overview

The second paper, *Real-Time Execution of Action Chunking Flow Policies*, directly addresses the latency problem [2]. It begins from the observation that modern generalist robot policies, especially diffusion- or flow-based VLAs, can be too slow for synchronous real-time control. Action chunking helps temporal consistency, but it does not eliminate latency. When a chunk ends before the next one is ready, the robot pauses. When a new chunk arrives late and replaces the old plan naively, the robot may jump to a different strategy.

RTC introduces an inference-time algorithm for asynchronous execution. The robot executes the current chunk while the model generates the next one. Crucially, the new generation is conditioned on the old chunk. Actions that are guaranteed to execute are frozen; actions in the future are inpainted. This makes the algorithm a scheduling method as much as a policy method.

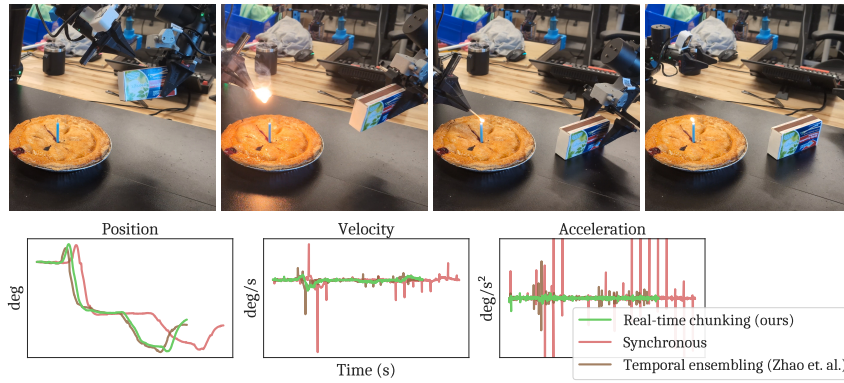


Figure 1: **Top:** Real-time chunking (RTC) enables the robot to perform highly dexterous and dynamic tasks, such as lighting a match—even in the presence of inference delays in excess of 300 milliseconds, corresponding to more than 30% of the model’s prediction horizon. **Bottom:** RTC performs the same robot motion 20% faster than synchronous inference [5, 30, 8, 24, 31, 59], and smoother than all competing methods, including temporal

Figure 4.1: RTC motivation figure, reproduced as a cropped paper figure. It contrasts real-time chunking with synchronous execution and emphasizes that smooth trajectories must be maintained even when inference delay is substantial.

4.2 The Latency Model

RTC formalizes the timing problem using a controller period Δt , a prediction horizon H , an execution horizon s , and an inference delay d . The policy produces A_t , but the result is

available only after d controller steps. If inference starts while the old chunk is executing, some of the old actions will be consumed before the new chunk can arrive. Those actions cannot be changed. They are committed.

This model is precisely what makes RTC relevant to an operating-systems course. There is a producer, the policy inference job, and a consumer, the robot controller. There is a buffer, the action queue. There is a deadline, the time by which the next chunk must be usable. There is also a correctness constraint: the produced chunk must be compatible with the actions already consumed.

Table 4.1: Runtime variables in RTC.

Symbol	Meaning
H	Prediction horizon: number of future actions generated by the policy.
s	Execution horizon: number of actions consumed before a new chunk is used.
d	Inference delay in controller steps.
A_t	Current predicted action chunk.
A'_t	Newly generated chunk that must be compatible with the already committed prefix.
W	Mask or guidance weight determining which positions are frozen and which are flexible.

4.3 Failure of Naive Asynchronous Execution

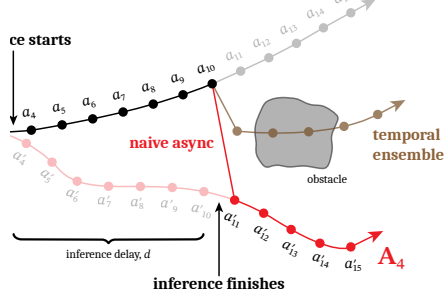
The simplest asynchronous strategy is to request the next chunk in the background and switch to it when it arrives. RTC shows why this can be worse than waiting. Flow and diffusion policies can represent multiple valid modes. One chunk may choose to go above an obstacle while the next chooses to go below it. If the system switches late from one mode to another, the resulting action sequence can contain a large discontinuity.

This failure mode is a scheduling failure with policy semantics. The runtime did overlap inference and execution, but it did not preserve compatibility between the output of the producer and the partially consumed buffer. The committed prefix must be visible to the generator. Otherwise the generator may produce a future that contradicts the robot's recent past.

4.4 Freeze and Inpaint

RTC solves the compatibility problem by posing real-time chunking as action inpainting. Suppose inference starts after some actions from the old chunk have already been scheduled. By the time the new chunk is ready, the first d overlapping actions are guaranteed to have been executed. RTC freezes these positions. The remaining positions are generated subject to guidance from the previous chunk.

termines the number of denoising steps.



An illustration of a typical bifurcation between chunks. Inference is started between a_4 and a_5 . The original chunk that was executing (black), had planned to go above the obstacle. However, a newly generated chunk $\{a'_t\}$ (red) goes below the obstacle. However, $\{a'_t\}$ is not available until d steps later. A naive asynchronous algorithm would switch from a_{10} to a'_{11} , inducing a very high, out-

Figure 4.2: RTC’s bifurcation example, reproduced as a cropped paper figure. A newly generated chunk can choose a different motion mode from the currently executing chunk, creating discontinuity when switched in late.

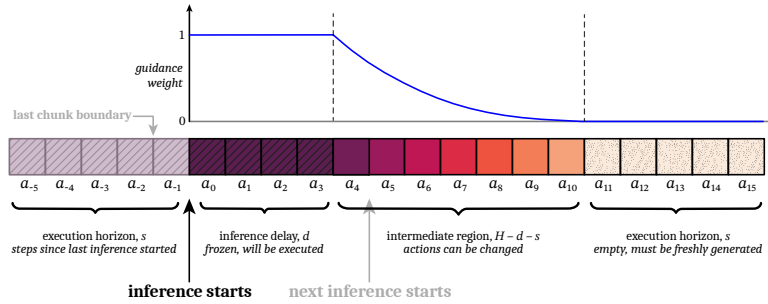


Figure 3: A diagram illustrating how action generation attends to the previous action chunk in real-time chunking. If inference starts after the execution of a_{-1} and the inference delay is $d = 4$, then the newly generated chunk will not be available until after a_3 is consumed. Therefore, $a_{0:3}$ are “frozen” and are attended to with a full guidance weight of 1. In the intermediate region, $a_{4:10}$, actions from the previous chunk are available but may be

Figure 4.3: RTC guidance diagram, reproduced as a cropped paper figure. The committed prefix receives full guidance, the intermediate overlap receives soft guidance, and the future suffix remains flexible.

The key point is that RTC treats the chunk as a partially constrained object. In image inpainting, known pixels are preserved while missing pixels are generated. In action inpainting, committed actions are preserved while future actions are sampled. The analogy is powerful because it translates runtime scheduling constraints into a form that flow-matching policies can use during sampling.

4.5 Soft Masking

Hard freezing only the first d positions may be insufficient. The new chunk may satisfy the committed prefix but still diverge quickly afterward. RTC therefore introduces soft masking across the overlap. Positions that are already committed receive weight 1. Positions far in the future receive weight 0. Intermediate positions receive a decaying guidance weight. This makes the transition smoother without forcing the entire future to duplicate the old chunk.

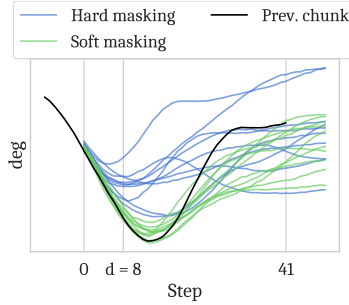


Figure 4: A comparison of naive inpainting (hard masking) and our proposed soft masking method: note that hard masking does not match the

Figure 4.4: Hard and soft masking in RTC, reproduced as a cropped paper figure. Soft masking preserves cross-chunk continuity more effectively than hard inpainting of the committed prefix alone.

Soft masking is a good example of a policy-aware scheduling decision. A purely real-time scheduler might only ensure that a new chunk arrives before the queue empties. RTC also asks whether the generated chunk is dynamically compatible with the old one. The mask schedule is therefore part of the scheduling policy.

4.6 Runtime Interpretation

RTC can be understood as a producer-consumer system with semantic constraints. The execution loop repeatedly consumes actions from the front of the queue. The inference loop observes the current state, reads the previous chunk, and produces a new chunk. The scheduler decides when to start inference and how to form the mask. If the delay is larger, the frozen region grows. If the execution horizon is smaller, the system updates more frequently but must handle more overlap.

Table 4.2: RTC as an operating-systems-style scheduling algorithm.

OS concept	RTC analogue
Producer	Background VLA or flow-policy inference job.
Consumer	Real-time robot controller consuming one action per tick.
Buffer	Action chunk queue.
Deadline	Next chunk must arrive before the queue starves.
Committed work	Prefix actions guaranteed to execute before inference completes.
Speculative work	Future suffix that can still be regenerated or in-painted.
Continuity constraint	New chunk must remain compatible with old chunk.

4.7 A Concrete Timeline Example

Assume a controller period of 20 ms and a flow policy that needs 160 ms to generate a chunk. This means $d = 8$ controller steps. Suppose the current chunk is

$$A = [a_0, a_1, \dots, a_{15}]$$

and inference starts immediately after a_1 is sent to the controller. By the time the new chunk is ready, the robot will already have executed approximately a_2 through a_9 . A naive new chunk A' cannot freely choose its first ten positions, because those positions correspond to a physical past that is already fixed. RTC therefore freezes the prefix and only allows the suffix to change gradually.

Table 4.3: Worked example of prefix commitment under delayed inference.

Time window	Runtime event	Status
$t = 0$ to $t = 1$	Old chunk starts executing and new inference is launched.	Old actions active
$t = 2$ to $t = 9$	Policy is still computing while controller consumes queued actions.	Prefix committed
$t = 10$	New chunk becomes available.	Switch point
$t = 10$ to $t = 15$	New chunk should remain compatible with old plan.	Soft overlap
$t > 15$	New suffix can follow updated observation more freely.	Mutable future

This example is the reason the word “committed” is useful. The scheduler is not merely predicting that certain actions are likely to execute; it knows they will execute unless the system applies an emergency stop. That knowledge should be fed into the generator. The scheduler therefore creates information that the model needs.

4.8 Algorithmic View

RTC can also be summarized as a loop. The controller pops an action from the queue at every tick. In parallel, the inference thread decides whether a new policy call should be launched. If no inference is running and the queue is approaching the execution horizon, the thread captures the current observation, computes the committed prefix length implied by expected delay, and calls the policy with an inpainting mask. When the result returns, the runtime merges the new chunk with the remaining queue.

masking procedure in GUIDEDINFERENCE, which explicitly defines a denoising function (Eq. 3) and computes a vector-Jacobian product, which can be done with reverse-mode autodifferentiation [2].

Algorithm 1 Real-Time Chunking

Require: flow policy π with prediction horizon H , minimum execution horizon $s_{\min}$, mutex $\mathcal{M}$, condition variable $\mathcal{C}$ associated with $\mathcal{M}$, initial chunk $\mathbf{A}_{\text{init}}$, initial delay estimate d_{init} , delay buffer size b , number of denoising steps n , maximum guidance weight β

```

1: procedure INITIALIZE_SHARED_STATE ▷ Initialize mutex-protected shared variables
2:    $t = 0$ ;  $\mathbf{A}_{\text{cur}} = \mathbf{A}_{\text{init}}$ ;  $\mathbf{o}_{\text{cur}} = \text{null}$ 

3: function GET_ACTION( $\mathbf{o}_{\text{next}}$ ) ▷ Called at an interval of  $\Delta t$  by controller
4:   with  $\mathcal{M}$  acquired do
5:      $t = t + 1$ 
6:      $\mathbf{o}_{\text{cur}} = \mathbf{o}_{\text{next}}$ 
7:     notify  $\mathcal{C}$ 
8:   return  $\mathbf{A}_{\text{cur}}[t - 1]$ 

```

Figure 4.5: RTC pseudocode area, reproduced as a cropped paper figure. The algorithmic view reinforces that RTC is a runtime loop coordinating controller ticks, inference calls, and queue updates.

4.9 Evaluation

RTC is evaluated in both simulation and real-world manipulation. In the Kinetix benchmark, dynamic tasks test whether policies can remain robust under injected inference delay. In real-world experiments, bimanual tasks such as lighting a match and manipulating objects test whether the method improves throughput and smoothness. The paper shows that RTC benefits not only wall-clock time but also progress measured in controller steps, suggesting that smoother and more frequent updates improve behavior beyond simply removing pauses.

4.10 Latency Cost

RTC is an inference-time method and does not require retraining. This is a practical advantage because it can be applied to existing flow-based policies. However, the method adds computation. The policy must incorporate guidance from the previous chunk and compute corrections during sampling. The paper reports latency measurements showing that RTC increases model inference time relative to vanilla sampling, although it remains substantially more efficient than some alternative batch-based methods.

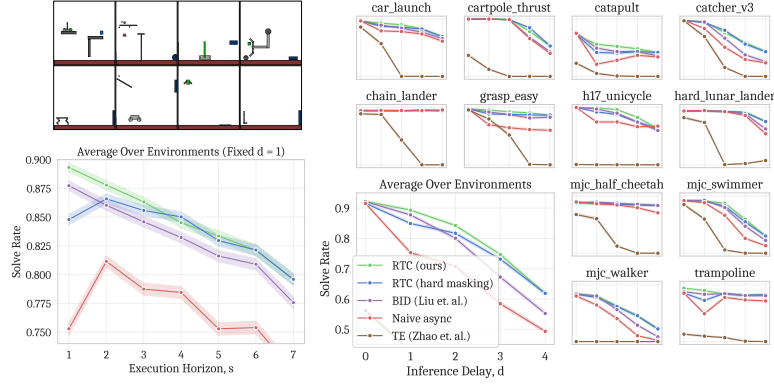


Figure 5: **Top left:** Kinetix environments; each involves getting a green object on the left to touch a blue one on the right. **Bottom left:** Execution horizon vs. solve rate with a fixed inference delay of 1. Only RTC and BID take full advantage of faster updates, showing strictly increasing performance with decreasing execution horizon. **Right:** Inference delay vs. solve rate with a fixed execution horizon of $s = \max(d, 1)$. RTC outperforms all baselines. Furthermore, soft masking (Sec. 3.2) improves performance at lower inference delays and execution horizons. Each data point represents 2048 trials, and 95% Wilson score intervals are shaded in.

Baselines. We compare against the following baselines:

Figure 4.6: RTC Kinetix evaluation, reproduced as a cropped paper figure. The plots show robustness to inference delay and improved use of shorter execution horizons compared with baselines.

- **TE, dense.** This strategy is the closest to the original TE in Zhao et al. [68]. We run inference as often as possible, resulting in $s = d$ for every chunk. This results in there always being at least 2 overlapping action chunks to ensemble, and often more.

We do not compare to BID [39] in the real world, as we found in simulation that it underperforms RTC while using significantly more compute—when applied to $\pi_{0.5}$ with a batch size of 16, BID has 2.3 times the latency of our method (see A.3 for latency measurements).

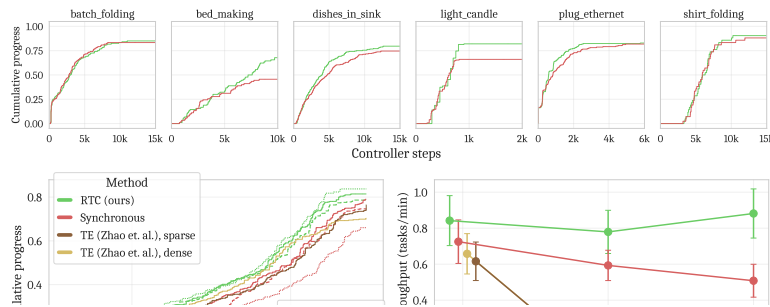


Figure 4.7: RTC real-world evaluation, reproduced as a cropped paper figure. The results compare cumulative task progress and throughput under synchronous, temporal ensembling, and real-time chunking execution.

Latency Measurements

Method	Latency
RTC (ours)	97ms
BID with $N = 16$ (no forward model)	115ms
BID with $N = 16$ (shared backbone)	169ms
BID with $N = 16$ (full)	223ms
Vanilla $\pi_{0.5}$	76ms

1: Latency measurements for various inference-time methods applied to $\pi_{0.5}$ [24]. Numbers include neural network inference only, and are averaged over 10 inference calls after 5 warmup calls. Inference is run on an NVIDIA RTX 4090 GPU using bfloat16 precision and $n = 5$ denoising steps. BID [39] slows down inference due to sampling batches of actions, whereas RTC slows down inference due to backpropagation through each denoising step. BID (no forward contrast) refers to a version of BID without the forward contrast which elides the need for a second model. BID (shared backbone) refers to a version of BID optimized for the π_0 architecture, where the VLM backbone (3B parameters) is shared between the streak model, so only two copies of the action expert (300M parameters) are needed. Full BID requires training of the entire model.

Component	Time (mobile)	Time (non-mobile)
Model	$96.89 \pm 0.16\text{ms}$	$97.43 \pm 0.28\text{ms}$
Network	$21.20 \pm 3.12\text{ms}$	$6.89 \pm 2.39\text{ms}$
Image resize	$11.22 \pm 5.00\text{ms}$	$1.44 \pm 0.27\text{ms}$
Other	$9.67 \pm 3.20\text{ms}$	$3.00 \pm 0.68\text{ms}$
Total	$138.98 \pm 6.71\text{ms}$	$108.76 \pm 2.34\text{ms}$

2: Breakdown of **total** inference latency by component for RTC. The image resizing component happens on the robot computer. In the mobile manipulation case, this computer is an Intel NUC portable computer with 12th Gen Intel i7-1260P processor. In the non-mobile case, this computer is a desktop workstation with AMD Ryzen 9 7950X processor. In both cases, the model runs on a separate workstation with an NVIDIA RTX 4090 GPU; the robot computer and the inference workstation are both connected to the same LAN via Ethernet connection, and communication happens via the WebSocket protocol. Model inference uses bfloat16 precision and $n = 5$ denoising steps. Measurements are taken from 50 inference calls during a single rollout, and $\pm$ one standard deviation is shown.

Component	Time (no RTC)	Time (with RTC)
Image encoders (SigLIP)	18ms	18ms
LLM prefill (Gemma 2B)	44ms	44ms
Denoising step (x5)	14ms	35ms
Total	76ms	97ms

3: Breakdown of **model** inference latency by component for vanilla $\pi_{0.5}$ and RTC. Measurements are taken from a single profiling trace for each method, run on an RTX 4090 GPU. RTC incurs a 2.5x latency increase per denoising step.

Figure 4.8: RTC latency tables, reproduced as a cropped appendix page. The tables show both the benefit and cost of RTC: it avoids execution stalls but adds inference-time overhead.

4.11 Contribution to the Thesis

RTC contributes the runtime scheduling layer. It takes ACT's action chunk and asks how to execute it when inference is too slow. The answer is not only to run inference in the background. The answer is to expose the prefix-suffix structure of the buffer and use it during generation. RTC therefore turns action chunking into a real-time scheduling primitive.

The paper also identifies the boundary of purely test-time repair. RTC can smooth chunk boundaries and hide latency without retraining, but it does not change how the policy was trained. If asynchronous execution creates observations that do not match the model's training assumptions, an inference-time mask may not be enough. This limitation motivates REMAC.

Paper 3: REMAC

5.1 Paper Overview

The third paper, *Real-Time Robot Execution with Masked Action Chunking*, begins where RTC leaves off [17]. It agrees that asynchronous inference is necessary for real-time robot execution and that inter-chunk discontinuity is a key problem. But it argues that another failure mode has been overlooked: intra-chunk inconsistency. During asynchronous execution, the current observation may correspond to a robot state produced by executing the prefix of an old chunk, while the newly generated chunk is conditioned on the current perception. The executed prefix and the perception can be partially misaligned with what the policy expects.

REMAC addresses this by adapting the policy with masked action chunking. Instead of assuming clean complete chunks during training, it exposes the model to partially masked or delayed chunks and trains it to make corrective continuations. It also introduces prefix-preserved sampling to strengthen continuity at inference time. The result is a method that targets both the training distribution and the runtime sampling procedure.

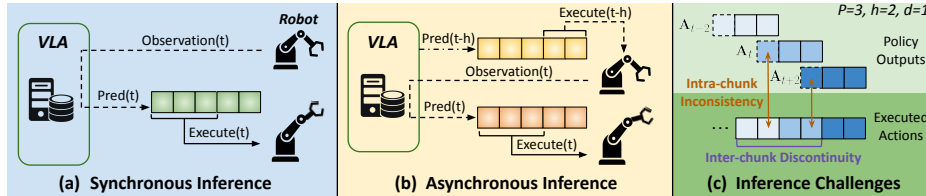


Figure 1: Illustration of execution paradigms. Arrowed lines of the same style indicate processes occurring simultaneously. **(a)** Synchronous inference: VLA prediction and robot execution alternate sequentially. **(b)** Asynchronous inference: VLA prediction runs concurrently with execution. **(c)** Although asynchronous inference enables real-time execution, it introduces two performance-degrading challenges: exacerbated inter-chunk discontinuity and intra-chunk inconsistency.

2.2 EXECUTION STRATEGIES FOR CHUNKED POLICIES

Figure 5.1: REMAC’s execution-paradigm figure, reproduced as a cropped paper figure. The figure distinguishes synchronous execution, asynchronous execution, inter-chunk discontinuity, and intra-chunk inconsistency.

5.2 Inter-Chunk and Intra-Chunk Failure Modes

RTC primarily focuses on inter-chunk discontinuity: the new chunk should connect smoothly to the old chunk. REMAC adds intra-chunk inconsistency: the current perception can be

inconsistent with the action prefix being executed. This is subtle but important. A policy trained on clean demonstrations may assume that the observation and the action sequence belong to the same coherent rollout. In asynchronous deployment, however, the prefix may come from an older observation and the suffix from a new one.

This mismatch is similar to a train-test distribution shift caused by the runtime scheduler. The scheduler changes when observations are sampled, which actions are committed, and how chunks overlap. If the model never sees such partially committed chunks during training, it may not learn how to correct them. REMAC therefore moves part of the scheduling problem into training.

5.3 Masked Action Chunking

REMAC trains the model with masked action chunks. The intuitive idea is to prevent the policy from relying on a clean, fully aligned prefix. Some parts of the action sequence are masked or corrupted to simulate asynchronous execution. The policy then learns to generate a continuation that is useful despite delay and mismatch. This is not merely data augmentation. It changes the model’s view of what information is reliable during deployment.

Table 5.1: Comparison between RTC and REMAC.

Dimension	RTC	REMAC
Primary level	Inference-time runtime algorithm	Training/adaptation plus sampling procedure
Main failure mode	Inter-chunk discontinuity	Inter-chunk discontinuity and intra-chunk inconsistency
Key mechanism	Freeze committed prefix, inpaint suffix	Masked action chunking and prefix-preserved sampling
Retraining required	No	Yes, typically lightweight adaptation
Latency impact	Adds inference-time compute	Designed not to add extra inference latency

5.4 Components of REMAC

REMAC contains several interacting components. Prefix masking shifts training attention away from action positions that would not directly influence execution under delay. A self-conditioned curriculum exposes the policy to its own predictions instead of only ground-truth chunks. The additional loss term encourages corrective adjustments that preserve useful temporal structure. Prefix-preserved sampling then ensures that deployment respects the committed actions.

A HARDWARE SETUP

Figure 6 illustrates our robot setup, where the gap between the gripper jaws is deliberately reduced to increase task difficulty and require finer control. Although two third-view cameras are mounted, only one third-view camera and the wrist-mounted camera are used for data processing. This configuration provides both a global view of the scene and a local, fine-grained perspective of the manipulation area, ensuring accurate observation while maintaining a controlled evaluation environment.



Figure 6: Robot setup illustration.

B SCHEDULING FUNCTION VARIANTS

Figure 7 illustrates different variants of the scheduling function σ . While multiple options are available, we primarily adopt the piecewise linear schedule, as it provides a smoother warm-up phase and more stable optimization compared to other choices.

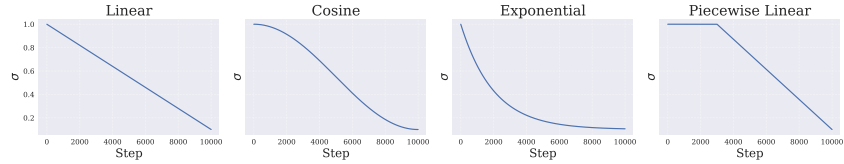


Figure 7: Different scheduling functions.

C TASK EXAMPLES

Figure 8 presents wrist-camera views of the tasks included in our experiments. These examples highlight the visual perspectives used for policy input and illustrate the varying levels of manipulation difficulty across tasks.

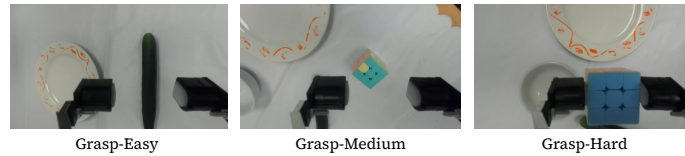


Figure 5.2: REMAC appendix figures showing robot setup, scheduling-function variants, and task examples. The cropped page adds deployment context for the real-world evaluation.

Table 5.2: REMAC components and their role in the thesis stack.

Component	Technical purpose	Thesis interpretation
Prefix masking	Avoid misleading supervision on stale prefix positions.	Train for scheduler-induced partial validity.
Self-conditioned curriculum	Expose the policy to its own asynchronous predictions.	Simulate deployment feedback during training.
Corrective loss	Encourage continuations that repair mismatch.	Treat the suffix as corrective control.
Prefix-preserved sampling	Keep committed actions fixed at inference time.	Align training with runtime commitment.

5.5 Prefix-Preserved Sampling

At inference time, REMAC preserves the committed prefix while generating the remaining suffix. This resembles RTC’s freeze-and-inpaint mechanism, but REMAC’s model has been trained to handle the masked structure. The prefix is not only a hard constraint imposed at deployment; it is part of the distribution the model has learned to condition on. This is the key reason REMAC can be interpreted as the robustness layer of the thesis.

5.6 Evaluation in Simulation

REMAC evaluates robustness under varying inference delays in Kinetix environments. The simulation setting is useful because delay can be controlled systematically. The results show that REMAC maintains higher solve rates as delay increases and often reduces execution time. The paper also reports ablations showing that the components of masked action chunking contribute to robustness.

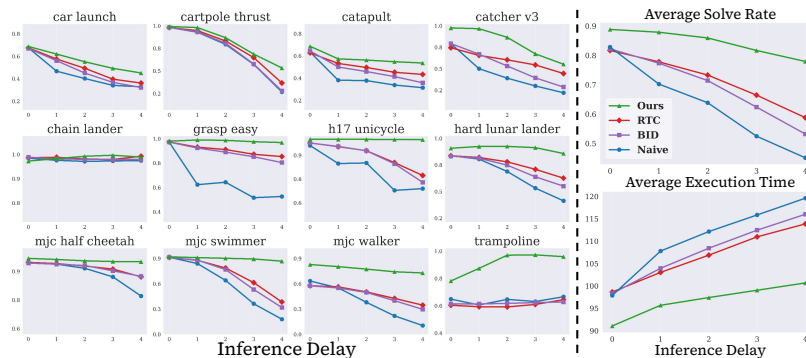


Figure 2: Performance comparison in Kinetix environments. **Left:** Solve rates for individual tasks under varying inference delays. **Right (top):** Average performance across all environments. **Our**

Figure 5.3: REMAC simulation results, reproduced as a cropped paper figure. The figure reports solve rates and execution time across Kinetix tasks under different inference delays.

Table 1: Effectiveness of the different components in REMAC.

Method	$d=0$	$d=1$	$d=2$	$d=3$	$d=4$
Naive	0.828	0.702	0.639	0.525	0.451
+ LoRA	0.825	0.710	0.630	0.510	0.428
+ Prefix masking	0.863	0.825	0.752	0.729	0.636
+ Self-conditioned curriculum	0.848	0.837	0.805	0.762	0.710
+ $\mathcal{L}_\Delta$ (Ours)	0.888	0.879	0.859	0.817	0.779

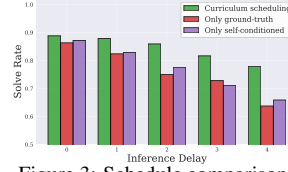


Figure 3: Schedule comparison.

tently outperforms all baselines across all delay settings, with especially pronounced gains under larger inference delays. It also achieves shorter average execution time than competing methods, demonstrating the ability to produce more efficient policies for faster task completion.

Our method also improves performance under $d = 0$. We attribute this to the masked action chunking strategy (REMAC), which encourages the policy to enforce stronger coherence and temporal dependencies even without inference delay, thereby enhancing its predictive ability. Moreover, the self-conditioned curriculum

Table 2: Integration with test-time methods.

Method	$d=0$	$d=1$	$d=2$	$d=3$	$d=4$
Ours	0.888	0.879	0.859	0.817	0.779
+ BID	0.888	0.880	0.862	0.821	0.781
+ RTC	0.888	0.879	0.864	0.826	0.791

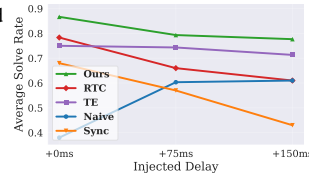
Figure 5.4: REMAC component ablations and integration with test-time methods, reproduced as a cropped paper figure. The table and schedule comparison show how masked action chunking improves delay robustness.

5.7 Evaluation on Real Robots

The real-world evaluation uses a Franka Research 3 robot with challenging grasp-and-place tasks. The tasks vary in difficulty and include tight grasping constraints. REMAC compares against synchronous inference, naive asynchronous execution, temporal ensembling, and RTC. The metric is average completion progress across task stages, which is appropriate because a trial may partially complete a long manipulation sequence even if it does not fully succeed.

Table 3: Average completion progress. Progress is measured by discrete scores corresponding to the sub-tasks completed.

Method	<i>Grasp-Easy</i>	<i>Grasp-Medium</i>	<i>Grasp-Hard</i>
Synchronous	0.805	0.718	0.670
Naive (Shukor et al., 2025)	0.825	0.825	0.460
TE (Zhao et al., 2023)	0.825	0.868	0.717
RTC (Black et al., 2025)	0.823	0.848	0.753
Ours	0.903	0.943	0.812

Figure 4: Performance under delay injections on *Grasp-Hard*.

sensors of the grippers are disabled and repurposed to narrow the gripping range, thereby acting as constraints for evaluating fine-grained control. This design ensures that the tasks demand precise action execution, making them well suited for assessing performance under asynchronous inference. For the backbone VLA, we use π_0 (Black et al., 2024), configured with a prediction horizon of $P = 50$, and adopt its memory-efficient variant for finetuning. A total of 200 trajectories are collected for model adaptation, with LoRA layers of rank 8 inserted only into the action expert module. After finetuning, the LoRA weights are merged into the backbone, ensuring that no additional computational overhead is introduced at inference.

Figure 5.5: REMAC real-robot completion progress and delay-injection result, reproduced as a cropped paper figure. The results show higher progress under delay, especially on the harder task.

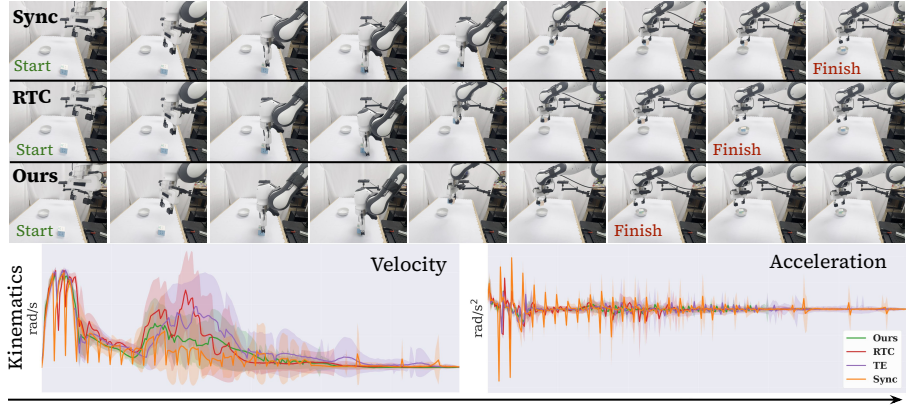


Figure 5: Visual comparison of different methods under a 150ms injected delay. **Top:** Task completion progress, where our method achieves faster completion. **Bottom:** Average robot kinematics during task execution on *Grasp-Hard*. Our method exhibits smoother trajectories and faster task completion speed.

significantly degraded performance. We attribute this to the fact that larger delays correspond to

Figure 5.6: REMAC visual and kinematic comparison under injected delay, reproduced as a cropped paper figure. The figure shows faster completion and smoother velocity/acceleration profiles.

5.8 Contribution to the Thesis

REMAC contributes the robustness layer. It shows that scheduling cannot be treated as a purely external runtime issue. The runtime changes the distribution of observations and action prefixes. Therefore, the policy must be trained or adapted to understand that some actions are committed and some are uncertain. This is a cross-layer conclusion: the scheduler and the policy must share an interface.

REMAC also clarifies a limitation of ACT and RTC. ACT assumes that chunking itself improves temporal coherence. RTC assumes that inference-time conditioning can preserve continuity. REMAC argues that asynchronous inference creates a deeper mismatch that must be addressed during learning. In a thesis on scheduling asynchronous inference, this is the final step: the scheduler’s behavior becomes part of the model’s training environment.

Performance Evaluation and Synthesis

6.1 Evaluation Methodology

The three papers use different evaluation methodologies because they answer different questions. ACT asks whether action chunking enables fine-grained imitation learning. RTC asks whether asynchronous execution can improve throughput and robustness to latency. REMAC asks whether the policy can remain reliable when asynchronous execution creates mismatch. A direct numerical comparison across all three papers would be misleading, because the tasks, robots, and policies differ. A useful synthesis instead compares the type of evidence each paper provides.

Table 6.1: Evaluation focus across the three main papers.

Paper	Main question	Primary metrics	Systems meaning
ACT	Can chunked imitation learn fine manipulation?	Success rate, ablations, temporal ensembling	Chunking improves policy stability.
RTC	Can slow chunking policies execute in real time?	Solve rate, cumulative progress, latency, smoothness	Runtime overlap and inpainting hide inference delay.
REMAC	Can policies tolerate asynchronous mismatch?	Solve rate, completion progress, execution time, kinematics	Training with masked chunks improves robustness.

6.2 Representation Versus Scheduling Versus Robustness

The main synthesis is that the three papers occupy different layers of the same problem. ACT changes the representation. RTC changes the runtime. REMAC changes the training distribution. Each layer is necessary but incomplete alone.

Representation without scheduling is not enough. ACT can produce smooth chunks, but if inference is slow, the robot can still pause at boundaries. Scheduling without robustness is not enough. RTC can maintain continuity at the boundary, but the model may still be surprised by observation-action mismatch induced by asynchronous execution. Robustness

without a schedulable representation is not enough. REMAC relies on the existence of chunks and prefixes; without chunking, there is no meaningful action mask.

6.3 A Unified Runtime Model

A unified model of asynchronous inference contains four state variables. First, the action queue Q stores executable commands. Second, the inference job J is either idle or running with a start observation o_τ . Third, the committed prefix C contains actions that will execute before J returns. Fourth, the mutable suffix M contains future actions that can be generated, inpainted, or replaced. The scheduler’s goal is to keep Q non-empty while ensuring that the transition from C to M remains smooth and semantically consistent.

Table 6.2: Unified scheduling state for action-chunking robot policies.

State variable	Description	Related paper
Q	Queue of pending actions consumed by the low-level controller.	ACT, RTC, SmolVLA
J	Background inference job producing a future chunk.	RTC, SmolVLA
C	Prefix guaranteed to be executed before the new chunk arrives.	RTC, REMAC
M	Future suffix that remains mutable and can be inpainted.	RTC, REMAC
o_t	Current perception used to condition generation.	All three

The scheduling policy decides when to launch J . A simple threshold policy launches inference when the queue length falls below a threshold. A more semantic policy might consider task phase, measured uncertainty, network latency, or expected need for reactivity. RTC can be seen as a semantic policy for forming C and M once J is launched. REMAC can be seen as a training policy for making the model robust to the distributions of C and M induced by the scheduler.

6.4 Performance Lessons

Several lessons emerge from the reported results. First, chunking improves stability and sample efficiency in fine manipulation. This is ACT’s main empirical message. Second, asynchronous execution improves wall-clock throughput when inference is slow, but naive asynchronous execution can introduce discontinuities. This is RTC’s main empirical message. Third, the degree of delay matters not only because it increases waiting time, but also because it enlarges the committed prefix and therefore the mismatch between old intent and current observation. This is REMAC’s main empirical message.

Table 6.3: Qualitative performance synthesis.

Criterion	ACT	RTC	REMAC
Fine manipulation	Strong bimanual evidence	Challenging bimanual tasks	Precision grasp-and-place tasks
Latency handling	Not the main target	Core contribution	Core contribution under mismatch
No retraining	Requires policy training	Test-time method	Requires adaptation or training
Continuity	Temporal ensembling	Freeze, inpaint, soft mask	Prefix-preserved sampling
Mismatch robustness	Limited	Partly addressed by conditioning	Explicitly trained

6.5 What the Evaluations Do Not Yet Resolve

The three papers provide strong evidence, but a thesis-level synthesis should also identify what remains unresolved. First, there is no single benchmark that evaluates representation, runtime scheduling, and training-time robustness together. ACT uses low-cost bimanual tasks, RTC uses Kinetix and bimanual flow-policy experiments, and REMAC uses Kinetix plus Franka grasp-and-place tasks. This diversity is valuable, but it prevents a clean end-to-end comparison.

Second, most results are reported at the policy-method level rather than the scheduler-policy interface level. For example, we often see success rate or completion progress, but not a complete trace of queue length, inference start times, delay estimates, committed-prefix length, and mask values. Such traces would make it possible to attribute a failure to queue starvation, mode bifurcation, poor observation timing, or learned-policy weakness.

Third, safety metrics remain underdeveloped. Smoothness is often measured through velocity or acceleration traces, but contact force, constraint violation, and near-collision rates are rarely central. For manipulation in homes, factories, or clinical environments, a scheduler that hides latency but occasionally produces a high-jerk transition may be unacceptable. Future evaluation should therefore include both task success and safety envelopes.

Finally, the papers mostly study single-robot or single-policy settings. A deployed robot may run perception, language grounding, navigation, manipulation, and safety filters concurrently. The action queue for manipulation may compete with other inference tasks for GPU time. This creates a second level of scheduling: not only how to schedule chunks within a policy, but how to schedule multiple policies and perception jobs sharing an accelerator.

6.6 A Proposed Evaluation Trace

A future benchmark for asynchronous inference should log a standardized trace. Each controller step should record the observation timestamp, action source chunk, queue length, whether inference is running, predicted inference completion time, committed-prefix length,

and any emergency smoothing or fallback action. Each policy call should record the input observation, previous chunk, mask schedule, denoising steps, and returned chunk. Each episode should report task progress, completion time, number of queue starvation events, maximum velocity and acceleration, and number of large boundary jumps.

Table 6.4: Proposed trace fields for evaluating asynchronous robot inference schedulers.

Trace field	Why it matters
Queue length over time	Detects starvation and excessive buffering.
Inference start and finish timestamps	Separates model latency from scheduling delay.
Committed-prefix length	Measures how much of a new chunk is constrained by old execution.
Mask or guidance weights	Explains how continuity is enforced.
Boundary velocity and acceleration	Captures motion quality at chunk switches.
Observation age	Measures stale perception under asynchronous execution.
Task phase label	Allows latency sensitivity to be analyzed by phase.

6.7 Scheduling Design Rules

The first design rule is to keep the action queue non-empty. This is the basic real-time requirement. If the queue starves, the robot pauses or must fall back to a hold action. Queue starvation is especially harmful in contact-rich manipulation because contact can be lost.

The second design rule is to separate committed and mutable actions. A runtime must know which actions are guaranteed to execute before the next inference result arrives. These actions form the prefix that cannot be contradicted. The remaining future forms the suffix that can be generated. RTC makes this rule explicit.

The third design rule is to expose scheduler-induced mismatch during training. If the model is deployed with asynchronous prefixes and delayed observations, it should see analogous conditions during training or adaptation. REMAC makes this rule explicit.

The fourth design rule is to measure both task success and motion quality. A method can improve wall-clock time by removing pauses but still produce jerky motion. Conversely, a method can smooth trajectories but reduce reactivity. Good evaluation therefore needs success, progress, latency, smoothness, and robustness to injected delay.

6.8 How the Three Papers Form a Thesis

The three papers form a coherent thesis because each paper contributes a missing piece of the same stack. ACT makes actions chunked. RTC makes chunked actions asynchronous.

REMAC makes asynchronous chunks robust. This statement is concise, but it captures a substantial cross-layer dependency.

ACT alone would be a thesis about imitation learning for fine manipulation. RTC alone would be a thesis about test-time execution of flow policies. REMAC alone would be a thesis about delay-robust policy adaptation. Together they become a thesis about scheduling asynchronous inference in robotic systems. The scheduling interface is the action chunk; the runtime problem is the production and consumption of chunks under delay; the learning problem is the distribution shift created by that runtime.

Chapter 7

Open Problems

7.1 Deadline-Aware Action Queue Scheduling

Current asynchronous systems often use simple thresholds for when to request a new chunk. A more principled scheduler would estimate inference delay, execution horizon, task phase, and uncertainty. For example, when the robot is approaching contact, the scheduler may prefer shorter execution horizons and more frequent updates. During a stable transport phase, it may prefer longer horizons and fewer inference calls. This resembles adaptive scheduling in operating systems, but the cost function includes physical smoothness and task progress rather than only CPU utilization.

7.2 Adaptive Prediction and Execution Horizons

ACT, RTC, and REMAC all use horizons as important hyperparameters. A fixed prediction horizon may not be optimal across a task. Long horizons provide stability but risk stale plans. Short horizons provide reactivity but increase inference frequency and overlap complexity. Future work should treat H and s as runtime-controlled variables. The policy could output chunks at multiple temporal resolutions, and the scheduler could select the appropriate resolution based on latency and task phase.

7.3 Multi-Policy and Multi-Rate Scheduling

Real robots often have multiple policies or controllers. A large VLA may provide high-level action chunks, while a fast local controller handles stabilization or contact reflexes. Scheduling asynchronous inference then becomes a multi-rate problem. The system must decide when to trust the large policy, when to use a local fallback, and how to merge commands. This is a natural extension of the committed-prefix idea: some actions are committed by the high-level policy, while lower-level controllers may still modify them for safety.

7.4 Observability and Runtime Debugging

Asynchronous inference failures can be difficult to diagnose. A task failure may come from model error, perception error, queue starvation, network delay, mode switching, or mismatch between observation and committed action prefix. Future systems should log queue length, inference start time, returned chunk timestamp, committed-prefix length,

mask weights, and motion smoothness. These logs would make scheduling failures visible in the same way that OS traces make CPU scheduling and I/O bottlenecks visible.

7.5 Safety and Formal Guarantees

The papers studied here are empirical. They show that chunking, inpainting, and masked training improve performance, but they do not provide formal safety guarantees. In safety-critical robotics, the scheduler may need to ensure that generated suffixes satisfy constraints on velocity, acceleration, collision, and contact force. A future thesis could combine learned chunk generation with control barrier functions or model predictive safety filters. The challenge is to preserve the flexibility of learned policies while guaranteeing that asynchronous updates do not create unsafe transitions.

Conclusion and Future Work

8.1 Summary of Contributions

This report studied scheduling asynchronous inference in robotic systems through three papers. ACT introduced action chunking for fine-grained bimanual manipulation. Its central contribution to this thesis is the action chunk as a temporal object. RTC used that object as a runtime buffer and proposed real-time chunking through freeze-and-inpaint. Its central contribution is the committed-prefix and mutable-suffix scheduling view. REMAC identified a deeper mismatch caused by asynchronous execution and proposed masked action chunking. Its central contribution is to make the policy robust to the scheduler-induced distribution shift.

The three papers therefore define a coherent trajectory. Action chunking begins as a policy representation trick. It becomes a scheduling interface when inference becomes slow. It finally becomes a training-time robustness problem when asynchronous scheduling changes what the policy sees at deployment.

8.2 Future Work

Future work should move toward cross-layer co-design. The policy should expose uncertainty, predicted duration, and chunk compatibility. The runtime should adapt inference timing and execution horizons. The training pipeline should simulate scheduler decisions and delay distributions. Evaluation should report not only final success rate but also latency, queue starvation, smoothness, and robustness to injected delays.

One promising direction is a scheduler-aware policy API. Instead of returning only an action chunk, the policy could return a chunk, confidence over each timestep, a suggested execution horizon, and a compatibility score with the previous chunk. The runtime could use this information to decide whether to accept, inpaint, shorten, or regenerate the chunk. Another direction is to train with randomized scheduler traces, so that the policy learns to handle a realistic distribution of delayed observations and committed prefixes.

8.3 Final Statement

The final lesson is simple. Robot control cannot assume that learning and systems are separable. When inference is slow, the runtime changes the data distribution. When actions are chunked, the policy exposes a schedulable buffer. When the buffer is consumed by a physical robot, prefix commitment becomes a semantic constraint. Scheduling asynchronous

inference in robotic systems is therefore a cross-layer problem: the operating system must understand action chunks, and the policy must understand the scheduler.

Bibliography

- [1] ALOHA project website. <https://tonyzhaozh.github.io/aloha/>
- [2] Kevin Black, Manuel Y. Galliker, and Sergey Levine. Real-Time Execution of Action Chunking Flow Policies. *arXiv:2506.07339*, 2025. <https://arxiv.org/abs/2506.07339>
- [3] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Joseph Dabis, Chelsea Finn, and others. RT-1: Robotics Transformer for Real-World Control at Scale. *arXiv:2212.06817*, 2022. <https://arxiv.org/abs/2212.06817>
- [4] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, Avinava Dubey, Chelsea Finn, and others. RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control. *arXiv:2307.15818*, 2023. <https://arxiv.org/abs/2307.15818>
- [5] Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion Policy: Visuomotor Policy Learning via Action Diffusion. *Robotics: Science and Systems*, 2023.
- [6] Danny Driess, Fei Xia, Mehdi S. M. Sajjadi, Corey Lynch, Aakanksha Chowdhery, Brian Ichter, Ayzaan Wahid, Jonathan Tompson, Quan Vuong, Tianhe Yu, Wenlong Huang, Yevgen Chebotar, and others. PaLM-E: An Embodied Multimodal Language Model. *arXiv:2303.03378*, 2023. <https://arxiv.org/abs/2303.03378>
- [7] Zipeng Fu, Tony Z. Zhao, and Chelsea Finn. Mobile ALOHA: Learning Bimanual Mobile Manipulation with Low-Cost Whole-Body Teleoperation. *arXiv:2401.02117*, 2024. <https://arxiv.org/abs/2401.02117>
- [8] Hugging Face LeRobot documentation. Real-Time Chunking (RTC). <https://huggingface.co/docs/lerobot/rtc>
- [9] Alexander Khazatsky, Karl Pertsch, Suraj Nair, Ashwin Balakrishna, Sudeep Dasari, Siddharth Karamcheti, Soroush Nasiriany, Mohan Kumar Srirama, Lawrence Yunliang Chen, Kirsty Ellis, and others. DROID: A Large-Scale In-The-Wild Robot Manipulation Dataset. *arXiv:2403.12945*, 2024. <https://arxiv.org/abs/2403.12945>
- [10] Moo Jin Kim et al. OpenVLA: An Open-Source Vision-Language-Action Model. *arXiv:2406.09246*, 2024. <https://arxiv.org/abs/2406.09246>
- [11] Yuejiang Liu, Jubayer Ibn Hamid, Annie Xie, Yoonho Lee, Maximilian Du, and Chelsea Finn. Bidirectional Decoding: Improving Action Chunking via Closed-Loop Resampling. *arXiv:2408.17355*, 2024. <https://arxiv.org/abs/2408.17355>

- [12] Octo Model Team. Octo: An Open-Source Generalist Robot Policy. 2024.
- [13] Open X-Embodiment Collaboration, Abby O’Neill, Abdul Rehman, Abhinav Gupta, Abhiram Maddukuri, Abhishek Gupta, Abhishek Padalkar, Abraham Lee, Acorn Pooley, Agrim Gupta, Ajay Mandlekar, and others. Open X-Embodiment: Robotic Learning Datasets and RT-X Models. *arXiv:2310.08864*, 2023. <https://arxiv.org/abs/2310.08864>
- [14] Physical Intelligence. Real-Time Chunking project page. https://pi.website/research/real_time_chunking
- [15] Nur Muhammad Mahi Shafiullah, Zichen Jeff Cui, Ariuntuya Altanzaya, and Lerrel Pinto. Behavior Transformers: Cloning k Modes with One Stone. *arXiv:2206.11251*, 2022. <https://arxiv.org/abs/2206.11251>
- [16] Mustafa Shukor et al. SmolVLA: A Vision-Language-Action Model for Affordable and Efficient Robotics. *arXiv:2506.01844*, 2025. <https://arxiv.org/abs/2506.01844>
- [17] Haoxuan Wang, Gengyu Zhang, Yan Yan, Yuzhang Shang, Ramana Rao Kompella, and Gaowen Liu. Real-Time Robot Execution with Masked Action Chunking. *ICLR 2026*, 2026. <https://openreview.net/forum?id=r0RGJ1j9on>
- [18] Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware. *arXiv:2304.13705*, 2023. <https://arxiv.org/abs/2304.13705>